



# Red Hat OpenShift AI Self-Managed 3.4

## Deploying models

Deploy models in Red Hat OpenShift AI Self-Managed



## Red Hat OpenShift AI Self-Managed 3.4 Deploying models

---

Deploy models in Red Hat OpenShift AI Self-Managed

## Legal Notice

Copyright © Red Hat.

Except as otherwise noted below, the text of and illustrations in this documentation are licensed by Red Hat under the Creative Commons Attribution–Share Alike 3.0 Unported license . If you distribute this document or an adaptation of it, you must provide the URL for the original version.

Red Hat, as the licensor of this document, waives the right to enforce, and agrees not to assert, Section 4d of CC-BY-SA to the fullest extent permitted by applicable law.

Red Hat, the Red Hat logo, JBoss, Hibernate, and RHCE are trademarks or registered trademarks of Red Hat, LLC. or its subsidiaries in the United States and other countries.

Linux<sup>®</sup> is the registered trademark of Linus Torvalds in the United States and other countries.

XFS is a trademark or registered trademark of Hewlett Packard Enterprise Development LP or its subsidiaries in the United States and other countries.

The OpenStack<sup>®</sup> Word Mark and OpenStack logo are trademarks or registered trademarks of the Linux Foundation, used under license.

All other trademarks are the property of their respective owners.

## Abstract

As a Red Hat OpenShift AI user, you can deploy your machine-learning models in Red Hat OpenShift AI Self-Managed.

## Table of Contents

|                                                                                 |           |
|---------------------------------------------------------------------------------|-----------|
| <b>CHAPTER 1. STORING MODELS</b>                                                | <b>3</b>  |
| 1.1. USING OCI CONTAINERS FOR MODEL STORAGE                                     | 3         |
| 1.2. STORING A MODEL IN AN OCI IMAGE                                            | 3         |
| 1.3. UPLOADING MODEL FILES TO A PERSISTENT VOLUME CLAIM (PVC)                   | 5         |
| <b>CHAPTER 2. DEPLOYING MODELS</b>                                              | <b>7</b>  |
| 2.1. AUTOMATIC SELECTION OF SERVING RUNTIMES                                    | 7         |
| 2.1.1. Hardware profile matching                                                | 7         |
| 2.1.2. Predictive model selection                                               | 7         |
| 2.1.3. Selection limitations                                                    | 7         |
| 2.1.4. Manual serving runtime selection                                         | 7         |
| 2.1.5. Administrator overrides                                                  | 7         |
| 2.2. DEPLOYMENT STRATEGIES FOR RESOURCE OPTIMIZATION                            | 7         |
| 2.2.1. Choosing a deployment strategy                                           | 8         |
| 2.3. DEPLOYING MODELS ON THE MODEL SERVING PLATFORM                             | 9         |
| 2.4. DEPLOYING MODELS BY USING THE MLSERVER RUNTIME                             | 13        |
| 2.5. DEPLOYING A MODEL STORED IN AN OCI IMAGE BY USING THE CLI                  | 16        |
| 2.6. MONITORING MODELS                                                          | 19        |
| 2.6.1. Viewing performance metrics for a deployed model                         | 19        |
| 2.6.2. Viewing model-serving runtime metrics for the model serving platform     | 20        |
| <b>CHAPTER 3. DEPLOYING MODELS ON THE NVIDIA NIM MODEL SERVING PLATFORM</b>     | <b>22</b> |
| 3.1. DEPLOYING MODELS ON THE NVIDIA NIM MODEL SERVING PLATFORM                  | 22        |
| 3.2. VIEWING NVIDIA NIM METRICS FOR A NIM MODEL                                 | 24        |
| 3.3. VIEWING PERFORMANCE METRICS FOR A NIM MODEL                                | 25        |
| <b>CHAPTER 4. MAKING INFERENCE REQUESTS TO DEPLOYED MODELS</b>                  | <b>27</b> |
| 4.1. ACCESSING THE AUTHENTICATION TOKEN FOR A DEPLOYED MODEL                    | 27        |
| 4.2. ACCESSING THE INFERENCE ENDPOINT FOR A DEPLOYED MODEL                      | 27        |
| 4.3. MAKING INFERENCE REQUESTS TO MODELS DEPLOYED ON THE MODEL SERVING PLATFORM | 28        |
| 4.4. INFERENCE ENDPOINTS                                                        | 28        |
| 4.4.1. Caikit TGIS ServingRuntime for KServe                                    | 28        |
| 4.4.2. OpenVINO Model Server                                                    | 28        |
| 4.4.3. vLLM NVIDIA GPU ServingRuntime for KServe                                | 29        |
| 4.4.4. vLLM Intel Gaudi Accelerator ServingRuntime for KServe                   | 30        |
| 4.4.5. vLLM AMD GPU ServingRuntime for KServe                                   | 30        |
| 4.4.6. vLLM Spyre AI Accelerator ServingRuntime for KServe                      | 30        |
| 4.4.7. vLLM Spyre s390x ServingRuntime for KServe                               | 30        |
| 4.4.8. vLLM Spyre ppc64le ServingRuntime for KServe                             | 30        |
| 4.4.9. NVIDIA Triton Inference Server                                           | 30        |
| 4.4.10. MLServer ServingRuntime for KServe                                      | 31        |
| 4.4.11. Additional resources                                                    | 32        |



# CHAPTER 1. STORING MODELS

You must store your model before you can deploy it. You can store a model in an S3 bucket, URI or Open Container Initiative (OCI) containers.

## 1.1. USING OCI CONTAINERS FOR MODEL STORAGE

As an alternative to storing a model in an S3 bucket or URI, you can upload models to Open Container Initiative (OCI) containers. Deploying models from OCI containers is also known as modelcars in KServe.

Using OCI containers for model storage can help you:

- Reduce startup times by avoiding downloading the same model multiple times.
- Reduce disk space usage by reducing the number of models downloaded locally.
- Improve model performance by allowing pre-fetched images.

Using OCI containers for model storage involves the following tasks:

- Storing a model in an OCI image.
- Deploying a model from an OCI image by using either the user interface or the command line interface. To deploy a model by using:
  - The user interface, see [Deploying models on the model serving platform](#).
  - The command line interface, see [Deploying a model stored in an OCI image by using the CLI](#).

## 1.2. STORING A MODEL IN AN OCI IMAGE

You can store a model in an OCI image. The following procedure uses the example of storing a MobileNet v2-7 model in ONNX format.

### Prerequisites

- You have a model in the ONNX format. The example in this procedure uses the MobileNet v2-7 model in ONNX format.
- You have installed the Podman tool.

### Procedure

1. In a terminal window on your local machine, create a temporary directory for storing both the model and the support files that you need to create the OCI image:

```
cd $(mktemp -d)
```

2. Create a **models** folder inside the temporary directory:

```
mkdir -p models/1
```

**NOTE**

This example command specifies the subdirectory **1** because OpenVINO requires numbered subdirectories for model versioning. If you are not using OpenVINO, you do not need to create the **1** subdirectory to use OCI container images.

3. Download the model and support files:

```
DOWNLOAD_URL=https://github.com/onnx/models/raw/main/validated/vision/classification/mobilenet/model/mobilenetv2-7.onnx
curl -L $DOWNLOAD_URL -O --output-dir models/1/
```

4. Use the **tree** command to confirm that the model files are located in the directory structure as expected:

```
tree
```

The **tree** command should return a directory structure similar to the following example:

```
.
├── Containerfile
└── models
    └── 1
        └── mobilenetv2-7.onnx
```

5. Create a Docker file named **Containerfile**:

**NOTE**

- Specify a base image that provides a shell. In the following example, **ubi9-micro** is the base container image. You cannot specify an empty image that does not provide a shell, such as **scratch**, because KServe uses the shell to ensure the model files are accessible to the model server.
- Change the ownership of the copied model files and grant read permissions to the root group to ensure that the model server can access the files. OpenShift runs containers with a random user ID and the root group ID.

```
FROM registry.access.redhat.com/ubi9/ubi-micro:latest
COPY --chown=0:0 models /models
RUN chmod -R a=rX /models

# nobody user
USER 65534
```

6. Use **podman build** commands to create the OCI container image and upload it to a registry. The following commands use Quay as the registry.

**NOTE**

If your repository is private, ensure that you are authenticated to the registry before uploading your container image.

```
podman build --format=oci -t quay.io/<user_name>/<repository_name>:<tag_name> .
podman push quay.io/<user_name>/<repository_name>:<tag_name>
```

### 1.3. UPLOADING MODEL FILES TO A PERSISTENT VOLUME CLAIM (PVC)

When deploying a model, you can serve it from a preexisting Persistent Volume Claim (PVC) where your model files are stored. You can upload your local model files to a PVC in the IDE that you access from a running workbench.

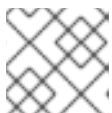
#### Prerequisites

- You have access to the OpenShift AI dashboard.
- You have access to a project that has a running workbench.
- You have created a persistent volume claim (PVC) with a context type of **Model storage**.
- The workbench is attached to the persistent volume (PVC).
  - For instructions on attaching a PVC, see [Creating a project workbench](#).
- You have the model files saved on your local machine.

#### Procedure

Follow these steps to upload your model files to the PVC mount point (**/opt/app-root/src/**) within your workbench:

1. From the OpenShift AI dashboard, click the open icon (  ) to open your IDE in a new window.
2. In your IDE, navigate to the **File Browser** pane on the left-hand side.
  - a. In JupyterLab, this is usually labeled **Files**.
  - b. In code-server, this is usually the **Explorer** view.
3. In the file browser, navigate to the **/opt/app-root/src/** folder. This folder represents the root of your attached PVC.



#### NOTE

Any files or folders that you create or upload to this folder persist in the PVC.

4. Optional: Create a new folder to organize your models:
  - a. In the file browser, right-click within the **/opt/app-root/src/** folder in the file browser and select **New Folder**.
  - b. Name the folder (for example, **models**).
  - c. Double-click the new **models** folder to enter it.
5. Upload your model files to the current folder (**/opt/app-root/src/** or **/opt/app-root/src/models/**):

- **Using JupyterLab:**

- a. Click the **Upload Files** icon (  ) in the file browser toolbar above the folder listing.
- b. In the file selection dialog, navigate to and select the model files from your local computer. Click **Open**.
- c. Wait for the upload progress bars next to the filenames to complete.

- **Using code-server:**

- a. Drag the model files directly from your local file explorer and drop them into the file browser pane in the target folder within code-server.
6. Wait for the upload process to complete.

## Verification

- Confirm that your files appear in the file browser at the path where you uploaded them.

## Next steps

When you follow the procedure to deploy a model, you can access the model files from the specified path within your PVC:

1. In the **Deploy model** dialog, select **Existing cluster storage** under the **Source model location** section.
2. From the **Cluster storage** list, select the PVC associated with your workbench.
3. In the **Model path** field, enter the path to your model or the folder containing your model.

## CHAPTER 2. DEPLOYING MODELS

The model serving platform is based on the KServe component and deploys each model from its own dedicated model server. This architecture is ideal for deploying, monitoring, scaling, and maintaining large models that require more resources, such as large language models (LLMs).

### 2.1. AUTOMATIC SELECTION OF SERVING RUNTIMES

When you deploy a model, OpenShift AI can automatically select the best serving runtime for your deployment. This feature allows you to efficiently deploy applications without needing to manually research runtime compatibility. The system determines the optimal runtime by analyzing the model type, model format, and selected hardware profile.

#### 2.1.1. Hardware profile matching

The system suggests a runtime by matching the accelerator defined in your selected hardware profile with available runtimes. For example, if you select a hardware profile that uses an NVIDIA GPU accelerator, the system filters for compatible runtimes, such as **vLLM NVIDIA GPU ServingRuntime for KServe**.



#### NOTE

Automatic selection is available only if a hardware profile exists for the specific accelerator that you want to use.

#### 2.1.2. Predictive model selection

For predictive models, you must select a **Model format** before the system can determine the appropriate serving runtime.

#### 2.1.3. Selection limitations

The **Auto-select** option is displayed only when the system can identify a single, distinct match. If multiple serving runtime templates are defined for the same accelerator, the system cannot determine the best option automatically, and the auto-select option is not displayed for that hardware profile. In such cases, you must manually select a runtime.

#### 2.1.4. Manual serving runtime selection

You can manually select a specific runtime from the **Serving runtime** list if the automatically selected option does not meet your needs. This option is useful when you require a specific version of a runtime or want to use a custom runtime that you have added to the platform. The **Serving runtime** list displays all global and project-scoped serving runtime templates available to you.

#### 2.1.5. Administrator overrides

Cluster administrator settings can override standard hardware profile matching. If the **Use distributed inference with llm-d by default when deploying generative models** option is enabled in the administrator settings, the system defaults to the **Distributed inference with llm-d** runtime, regardless of other potential matches. This option is available in **Settings > Cluster settings > General settings**.

### 2.2. DEPLOYMENT STRATEGIES FOR RESOURCE OPTIMIZATION

To optimize resource usage and manage downtime during model rollouts, you can configure the deployment strategy for your inference services. Choosing the appropriate strategy depends on your cluster's available quotas, especially hardware accelerators such as GPUs, and your tolerance for service interruptions.

There are two primary deployment strategies available for model serving:

### Rolling update

This strategy ensures zero downtime and continuous availability of the model. New inference service pods start while the existing pods are running. Traffic is switched to the new pods only after they are fully ready, and then the old pods are terminated.

However, rolling updates require increased resources like CPU, memory, and GPUs during the update process. Plan for approximately 200% of the pod requests as headroom during the transition because parallel instances exist briefly.

### Recreate

This strategy prioritizes resource conservation over availability. All existing inference service pods are terminated before the new pods attempt to launch.

However, this method requires a period of downtime. The model endpoint is unavailable and returns errors between the termination of the old pod and the readiness of the new pod.

## 2.2.1. Choosing a deployment strategy

Choose the deployment strategy that best fits your availability requirements and resource quotas. The following table compares the rolling update and recreate strategies.

| Strategy              | Description                                                                                                    | Resource impact                                                                                                     | Recommended scenarios                                                                                                                                                                                                                                                 |
|-----------------------|----------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Rolling update</b> | Replaces pods gradually to ensure zero downtime. Traffic switches to new pods only after they are fully ready. | <b>High:</b> Requires approximately 200% of the request resources to host parallel instances during the transition. | <ul style="list-style-type: none"><li>● <b>Production workloads:</b> Environments where the model must remain accessible without interruption.</li><li>● <b>High-quota clusters:</b> Namespaces with sufficient headroom to accommodate parallel instances.</li></ul> |

| Strategy        | Description                                                                                       | Resource impact                                                                              | Recommended scenarios                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
|-----------------|---------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Recreate</b> | Terminates the old pod before starting the new one. Service is unavailable during the transition. | <b>Low:</b> Consumption does not exceed 100%. Prevents <i>Insufficient Resources</i> errors. | <ul style="list-style-type: none"> <li>● <b>Resource-constrained environments:</b> Projects using scarce hardware, such as high-end GPUs, where double allocation is not possible.</li> <li>● <b>Development and staging:</b> Environments where downtime does not impact business operations.</li> <li>● <b>Batch processing:</b> Workflows where immediate availability is not critical.</li> <li>● <b>Maintenance windows:</b> Periods where service unavailability is expected.</li> </ul> |



### IMPORTANT

The **Recreate** strategy severs the connection to the old pod immediately. Ensure that your traffic routing gateway and client applications can handle a temporary gap in service before applying this strategy.



### NOTE

The **Recreate** deployment strategy is available for all runtimes except **Distributed inference with llm-d**. If you select the **Distributed inference with llm-d** runtime, the deployment strategy options are not displayed and the system defaults to the Recreate strategy.

## 2.3. DEPLOYING MODELS ON THE MODEL SERVING PLATFORM

You can deploy generative AI (gen AI) or predictive AI models on the model serving platform by using the **Deploy a model** wizard. The wizard allows you to configure your model, including specifying its location and type, selecting a serving runtime, assigning a hardware profile, and setting advanced configurations like external routes and token authentication.

To successfully deploy a model, you must meet the following prerequisites.

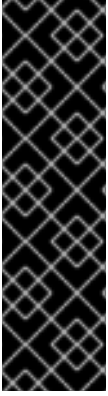
### General prerequisites

- You have logged in to Red Hat OpenShift AI.
- You have installed KServe and enabled the model serving platform.
- You have enabled a preinstalled or custom model-serving runtime.
- You have created a project.
- You have access to S3-compatible object storage, a URI-based repository, an OCI-compliant registry or a persistent volume claim (PVC) and have added a connection to your project. For more information about adding a connection, see [Adding a connection to your project](#).
- If you want to use graphics processing units (GPUs) with your model server, you have enabled GPU support in OpenShift AI. If you use NVIDIA GPUs, see [Enabling NVIDIA GPUs](#). If you use AMD GPUs, see [AMD GPU integration](#).

## Runtime-specific prerequisites

Meet the requirements for the specific runtime you intend to use.

- **Caikit-TGIS runtime**
  - To use the Caikit-TGIS runtime, you have converted your model to Caikit format. For an example, see [Converting Hugging Face Hub models to Caikit format](#) in the [caikit-tgis-serving](#) repository.
- **vLLM NVIDIA GPU ServingRuntime for KServe**
  - To use the **vLLM NVIDIA GPU ServingRuntime for KServer**runtime, you have enabled GPU support in OpenShift AI and have installed and configured the Node Feature Discovery Operator on your cluster. For more information, see [Installing the Node Feature Discovery Operator](#) and [Enabling NVIDIA GPUs](#).
- **vLLM CPU ServingRuntime for KServe**
  - To use the VLLM runtime on IBM Z and IBM Power, use the **vLLM CPU ServingRuntime for KServe**. You cannot use GPU accelerators with IBM Z and IBM Power architectures. For more information, see [Red Hat OpenShift Multi Architecture Component Availability Matrix](#).
- **vLLM Intel Gaudi Accelerator ServingRuntime for KServe**
  - To use the **vLLM Intel Gaudi Accelerator ServingRuntime for KServer**runtime, you have enabled support for hybrid processing units (HPUs) in OpenShift AI. This includes installing the Intel Gaudi Base Operator and configuring a hardware profile. For more information, see [Intel Gaudi Base Operator OpenShift installation](#) in the AMD documentation and [Working with hardware profiles](#).
- **vLLM AMD GPU ServingRuntime for KServe**
  - To use the **vLLM AMD GPU ServingRuntime for KServer**runtime, you have enabled support for AMD graphic processing units (GPUs) in OpenShift AI. This includes installing the AMD GPU operator and configuring a hardware profile. For more information, see [Deploying the AMD GPU operator on OpenShift](#) and [Working with hardware profiles](#).
- **vLLM Spyre AI Accelerator ServingRuntime for KServe**



## IMPORTANT

Support for IBM Spyre AI Accelerators on x86 is currently available in Red Hat OpenShift AI 3.4 as a Technology Preview feature. Technology Preview features are not supported with Red Hat production service level agreements (SLAs) and might not be functionally complete. Red Hat does not recommend using them in production. These features provide early access to upcoming product features, enabling customers to test functionality and provide feedback during the development process.

For more information about the support scope of Red Hat Technology Preview features, see [Technology Preview Features Support Scope](#).



## IMPORTANT

Support for IBM Spyre AI Accelerators on ppc64le is currently available in Red Hat OpenShift AI 3.4 as a General Availability (GA) feature.

- To use the **vLLM Spyre AI Accelerator ServingRuntime for KServer**runtime on x86, you have installed the Spyre Operator and configured a hardware profile. For more information, see [Spyre operator image](#) and [Working with hardware profiles](#).
- To use the **vLLM Spyre AI Accelerator ServingRuntime for KServer**runtime on ppc64le, you have installed the Spyre Operator and configured a hardware profile. For more information, see [Spyre operator image](#) and [Working with hardware profiles](#).
  - **vLLM Spyre s390x ServingRuntime for KServe**
- To use the **vLLM Spyre s390x ServingRuntime for KServer**runtime on IBM Z, you have installed the Spyre Operator and configured a hardware profile. For more information, see [Spyre operator image](#) and [Working with hardware profiles](#).

## Procedure

1. In the left menu, click **Projects**.
2. Click the name of the project that you want to deploy a model in.  
A project details page opens.
3. Click the **Deployments** tab.
4. Click **Deploy model**.  
The **Deploy a model** wizard opens.
5. In the **Model details** section, provide information about the model:
  - a. From the **Model location** list, specify where your model is stored and complete the connection detail fields.

**NOTE**

- The **OCI-compliant registry**, **S3 compatible object storage**, and **URI** options are preinstalled connection types. Additional options might be available if your OpenShift AI administrator added them.
- If you have uploaded model files to a persistent volume claim (PVC) and the PVC is attached to your workbench, the **Cluster storage** option becomes available in the **Model location** list. Use this option to select the PVC and specify the path to the model file.

- b. From the **Model type** list, select the type of model that you are deploying, **Predictive** or **Generative AI model**.
  - c. Click **Next**.
6. In the **Model deployment** section, configure the deployment:
  - a. In the **Model deployment name** field, enter a unique name for your model deployment.
  - b. In the **Description** field, enter a description of your deployment.
  - c. From the **Hardware profile** list, select a hardware profile.
  - d. Optional: To modify the default resource allocation, click **Customize resource requests and limits** and enter new values for the CPU and Memory requests and limits.
  - e. In the **Serving runtime** field, select one of the following options:
    - **Auto-select the best runtime for your model based on model type, model format, and hardware profile**  
The system analyzes the selected model framework and your available hardware profiles to recommend a serving runtime.
    - **Select from a list of serving runtimes, including custom ones**  
Select this option to manually choose a runtime from the list of global and project-scoped serving runtime templates.

For more information about how the system determines the best runtime and administrator overrides, see [Automatic selection of serving runtimes](#).
  - f. Optional: If you selected a **Predictive model** type, select a framework from the **Model framework (name - version)** list. This field is hidden for Generative AI models.
  - g. In the **Number of model server replicas to deploy** field, specify a value.
  - h. Click **Next**.
7. In the **Advanced settings** section, configure advanced options:
  - a. Optional: (Generative AI models only) Select the **Add as AI asset endpoint** checkbox if you want to add your model's endpoint to the **Gen AI studio → AI asset endpoints** page.
    - i. In the **Use case** field, enter the types of tasks that your model performs, such as chat, multimodal, or natural language processing.

**NOTE**

You must add your model as an AI asset endpoint to test your model on the **Gen AI studio** → **playground** page.

If you enabled the endpoint, enter the types of tasks that your model performs in the **Use case** field.

- b. Optional: Select the **Model access** checkbox to make your model deployment available through an external route.
- c. Optional: To require token authentication for inference requests to the deployed model, select **Require token authentication**.
- d. In the **Service account name** field, enter the service account name that the token will be generated for.
- e. To add an additional service account, click **Add a service account** and enter another service account name.
- f. Optional: Select **Add custom runtime arguments** or **Add custom runtime environment variables** to add configuration parameters to your deployment.
- g. In the **Deployment strategy** section, select **Rolling update** or **Recreate**. For more information about deployment strategies, see [Deployment strategies for resource optimization](#).

**NOTE**

The **Recreate** deployment strategy is available for all runtimes except **Distributed inference with llm-d**. If you select the **Distributed inference with llm-d** runtime, the deployment strategy options are not displayed and the system defaults to the **Recreate** strategy.

8. Click **Deploy**.

**Verification**

- Confirm that the deployed model is shown on the **Deployments** tab for the project, and on the **Deployments** page of the dashboard with a checkmark in the **Status** column.

**Additional resources**

- [Model-serving runtimes for accelerators](#)
- [Adding a custom model-serving runtime](#).
- [Deployment strategies for resource optimization](#).

**2.4. DEPLOYING MODELS BY USING THE MLSERVER RUNTIME**

Deploy models with the **MLServer ServingRuntime for KServe** option by selecting your model framework in the **Deploy a model** wizard. KServe automatically configures the runtime based on your model framework selection. If your model file uses a well-known filename and is located directly under **/mnt/models**, KServe automatically detects and loads it without additional configuration.

## Prerequisites

- You have logged in to Red Hat OpenShift AI.
- You have installed KServe and enabled the model serving platform.
- The **MLServer ServingRuntime for KServe** is enabled in your cluster. For more information, see [Enabling the model serving platform](#).
- You have created a project.
- Your model is stored in a location accessible to the model server and you have added a connection to your project:
  - OCI registry
  - S3-compatible object storage
  - URI
  - Persistent Volume Claim
- You are deploying a model that uses one of the supported MLServe implementations:
  - LightGBM
  - ONNX
  - Scikit-learn
  - XGBoost
- Your model file uses a well-known filename and is stored directly under **/mnt/models** after download. If your model uses a different filename or is stored in a subdirectory, you must manually configure the **MLSERVER\_MODEL\_URI** environment variable. Well-known filenames:
  - LightGBM: **model.bst**
  - ONNX: **model.onnx**
  - Scikit-learn: **model.joblib**, **model.pickle**, **model.pkl**
  - XGBoost: **model.bst**, **model.json**, **model.ubj**



### NOTE

KServe automatically configures the MLServe runtime environment variables based on your model framework selection. The model name is set to your model deployment name, the model implementation is set based on your selected framework, and the model URI is set to **/mnt/models**.



## IMPORTANT

You can also use MLServer's **model-settings.json** file for model configuration. If a **model-settings.json** file is present alongside your model file, the MLServer runtime loads configuration values from that file and overrides the automatic configuration. The model name in the file must match your model deployment name.

Configuration beyond model name, model URI, and model implementation is tested and verified, but not officially supported.

## Procedure

1. In the OpenShift AI dashboard, navigate to **Projects** and select or create a project.
2. Deploy the model using the **Deploy a model** wizard.  
For complete deployment instructions, see [Deploying models on the model serving platform](#).
3. In the **Model details** section of the wizard:
  - a. Select your model location from an existing data connection or cluster storage.
  - b. For **Model type**, select **Predictive**.
4. In the **Model deployment** section:
  - a. Enter a model deployment name and optional description.
  - b. Select your model framework from the **Model framework** list:
    - LightGBM
    - ONNX
    - Scikit-learn
    - XGBoost

The **Deployment resource** field automatically selects **MLServer ServingRuntime for KServe** based on your framework selection.
  - c. Configure the number of model server replicas to deploy.
5. If your model file does not use a well-known filename or is not located directly under **/mnt/models**, configure the **MLSERVER\_MODEL\_URI** environment variable in the **Advanced settings** section:
  - a. Select the **Add custom runtime environment variables** checkbox.
  - b. Click **Add variable**.
  - c. Add the **MLSERVER\_MODEL\_URI** variable with the full path to your model file, for example, **/mnt/models/my-custom-model.pkl** or **/mnt/models/subfolder/model.pkl**.
6. Click **Deploy** to deploy your model.

## Verification

- Confirm that the deployed model is shown on the **Deployments** tab for the project.

The deployment status displays as **Pending** during the deployment process. When the deployment completes successfully, the status changes to **Started**.

- Test the model by making an inference request:
  - a. On the **Deployments** tab, click the deployment name to view details.
  - b. Click **Internal and external endpoint** to display the inference endpoints.
  - c. Copy the external endpoint URL and query the model:

```
$ curl -X POST <inference_endpoint_url>/v2/models/<model_name>/infer \
-H "Content-Type: application/json" \
-d '{"inputs":[{"name":"<input_name>","shape":[<shape>],"datatype":"<datatype>","data":[<data>}]}'
```

where:

**<inference\_endpoint\_url>**

Specifies the external endpoint URL for your deployed model.

**<model\_name>**

Specifies the name of your deployed model.

**<input\_name>**

Specifies the input name expected by your model.

**<shape>**

Specifies the shape of your input data.

**<datatype>**

Specifies the data type, for example, **FP32**.

**<data>**

Specifies your input data values.

The model returns an inference response with the prediction results.

#### Additional resources

- [Inference endpoints](#)
- [Deploying models on the model serving platform](#)
- [MLServer LightGBM Runtime](#)
- [MLServer ONNX Runtime](#)
- [MLServer scikit-learn Runtime](#)
- [MLServer XGBoost Runtime](#)
- [MLServer OpenAPI / Inference API](#)

## 2.5. DEPLOYING A MODEL STORED IN AN OCI IMAGE BY USING THE CLI

You can deploy a model that is stored in an OCI image from the command line interface.

The following procedure uses the example of deploying a MobileNet v2-7 model in ONNX format, stored in an OCI image on an OpenVINO model server.



## NOTE

By default in KServe, models are exposed outside the cluster and not protected with authentication.

## Prerequisites

- You have stored a model in an OCI image as described in [Storing a model in an OCI image](#).
- If you want to deploy a model that is stored in a private OCI repository, you must configure an image pull secret. For more information about creating an image pull secret, see [Using image pull secrets](#).
- You are logged in to your OpenShift cluster.

## Procedure

1. Create a project to deploy the model:

```
oc new-project oci-model-example
```

2. Use the OpenShift AI Applications project **kserve-ovms** template to create a **ServingRuntime** resource and configure the OpenVINO model server in the new project:

```
oc process -n redhat-ods-applications -o yaml kserve-ovms | oc apply -f -
```

3. Verify that the **ServingRuntime** named **kserve-ovms** is created:

```
oc get servingruntimes
```

The command should return output similar to the following:

| NAME        | DISABLED | MODELTYPE   | CONTAINERS       | AGE |
|-------------|----------|-------------|------------------|-----|
| kserve-ovms |          | openvino_ir | kserve-container | 1m  |

4. Create an **InferenceService** YAML resource, depending on whether the model is stored from a private or a public OCI repository:
  - For a model stored in a public OCI repository, create an **InferenceService** YAML file with the following values, replacing **<user\_name>**, **<repository\_name>**, and **<tag\_name>** with values specific to your environment:

```
apiVersion: serving.kserve.io/v1beta1
kind: InferenceService
metadata:
  name: sample-isvc-using-oci
spec:
  predictor:
    model:
```

```

    runtime: kserve-ovms # Ensure this matches the name of the ServingRuntime
resource
  modelFormat:
    name: onnx
  storageUri: oci://quay.io/<user_name>/<repository_name>:<tag_name>
  resources:
    requests:
      memory: 500Mi
      cpu: 100m
      # nvidia.com/gpu: "1" # Only required if you have GPUs available and the model
and runtime will use it
    limits:
      memory: 4Gi
      cpu: 500m
      # nvidia.com/gpu: "1" # Only required if you have GPUs available and the model
and runtime will use it

```

- For a model stored in a private OCI repository, create an **InferenceService** YAML file that specifies your pull secret in the **spec.predictor.imagePullSecrets** field, as shown in the following example:

```

apiVersion: serving.kserve.io/v1beta1
kind: InferenceService
metadata:
  name: sample-isvc-using-private-oci
spec:
  predictor:
    model:
      runtime: kserve-ovms # Ensure this matches the name of the ServingRuntime
resource
  modelFormat:
    name: onnx
  storageUri: oci://quay.io/<user_name>/<repository_name>:<tag_name>
  resources:
    requests:
      memory: 500Mi
      cpu: 100m
      # nvidia.com/gpu: "1" # Only required if you have GPUs available and the model
and runtime will use it
    limits:
      memory: 4Gi
      cpu: 500m
      # nvidia.com/gpu: "1" # Only required if you have GPUs available and the model
and runtime will use it
    imagePullSecrets: # Specify image pull secrets to use for fetching container images,
including OCI model images
      - name: <pull-secret-name>

```

After you create the **InferenceService** resource, KServe deploys the model stored in the OCI image referred to by the **storageUri** field.

## Verification

Check the status of the deployment:

```
oc get inferencervice
```

■

The command should return output that includes information, such as the URL of the deployed model and its readiness state.

## 2.6. MONITORING MODELS

You can monitor models that are deployed on the model serving platform to view performance and resource usage metrics.

### 2.6.1. Viewing performance metrics for a deployed model

You can monitor the following metrics for a specific model that is deployed on the model serving platform:

- **Number of requests** - The number of requests that have failed or succeeded for a specific model.
- **Average response time (ms)** - The average time it takes a specific model to respond to requests.
- **CPU utilization (%)** - The percentage of the CPU limit per model replica that is currently utilized by a specific model.
- **Memory utilization (%)** - The percentage of the memory limit per model replica that is utilized by a specific model.

You can specify a time range and a refresh interval for these metrics to help you determine, for example, when the peak usage hours are and how the model is performing at a specified time.

#### Prerequisites

- You have installed Red Hat OpenShift AI.
- A cluster admin has enabled user workload monitoring (UWM) for user-defined projects on your OpenShift cluster. For more information, see [Enabling monitoring for user-defined projects](#) and [Configuring monitoring for the model serving platform](#).
- You have logged in to Red Hat OpenShift AI.
- The following dashboard configuration options are set to the default values as shown:

```
disablePerformanceMetrics:false
disableKServeMetrics:false
```

For more information about setting dashboard configuration options, see [Customizing the dashboard](#).

- You have deployed a model on the model serving platform by using a preinstalled runtime.



#### NOTE

Metrics are only supported for models deployed by using a preinstalled model-serving runtime or a custom runtime that is duplicated from a preinstalled runtime.

## Procedure

1. From the OpenShift AI dashboard navigation menu, click **Projects**.  
The **Projects** page opens.
2. Click the name of the project that contains the data science models that you want to monitor.
3. In the project details page, click the **Deployments** tab.
4. Select the model that you are interested in.
5. On the **Endpoint performance** tab, set the following options:
  - **Time range** - Specifies how long to track the metrics. You can select one of these values: 1 hour, 24 hours, 7 days, and 30 days.
  - **Refresh interval** - Specifies how frequently the graphs on the metrics page are refreshed (to show the latest data). You can select one of these values: 15 seconds, 30 seconds, 1 minute, 5 minutes, 15 minutes, 30 minutes, 1 hour, 2 hours, and 1 day.
6. Scroll down to view data graphs for number of requests, average response time, CPU utilization, and memory utilization.

## Verification

The **Endpoint performance** tab shows graphs of metrics for the model.

### 2.6.2. Viewing model-serving runtime metrics for the model serving platform

When a cluster administrator has configured monitoring for the model serving platform, non-admin users can use the OpenShift web console to view model-serving runtime metrics for the KServe component.

## Prerequisites

- A cluster administrator has configured monitoring for the model serving platform.
- You have been assigned the **monitoring-rules-view** role. For more information, see [Granting users permission to configure monitoring for user-defined projects](#).
- You are familiar with how to monitor project metrics in the OpenShift web console. For more information, see [Monitoring your project metrics](#).

## Procedure

1. Log in to the OpenShift web console.
2. Switch to the **Developer** perspective.
3. In the left menu, click **Observe**.
4. As described in [Monitoring your project metrics](#), use the web console to run queries for model-serving runtime metrics. You can also run queries for metrics that are related to OpenShift Service Mesh. Some examples are shown.
  - a. The following query displays the number of successful inference requests over a period of time for a model deployed with the vLLM runtime:

```
sum(increase(vllm:request_success_total{namespace=${namespace},model_name=${model_name}}[${rate_interval}]))
```



#### NOTE

Certain vLLM metrics are available only after an inference request is processed by a deployed model. To generate and view these metrics, you must first make an inference request to the model.

- b. The following query displays the number of successful inference requests over a period of time for a model deployed with the OpenVINO Model Server runtime:

```
sum(increase(ovms_requests_success{namespace=${namespace},name=${model_name}}[${rate_interval}]))
```

#### Additional resources

- [OVMS metrics](#)

## CHAPTER 3. DEPLOYING MODELS ON THE NVIDIA NIM MODEL SERVING PLATFORM

You can deploy models using NVIDIA NIM inference services on the NVIDIA NIM model serving platform.

NVIDIA NIM, part of NVIDIA AI Enterprise, is a set of microservices designed for secure, reliable deployment of high performance AI model inferencing across clouds, data centers and workstations.

### 3.1. DEPLOYING MODELS ON THE NVIDIA NIM MODEL SERVING PLATFORM

When you have enabled the **NVIDIA NIM model serving platform**, you can start to deploy NVIDIA-optimized models on the platform.

#### Prerequisites

- You have logged in to Red Hat OpenShift AI.
- You have enabled the **NVIDIA NIM model serving platform**
- You have created a project.
- You have enabled support for graphic processing units (GPUs) in OpenShift AI. This includes installing the Node Feature Discovery Operator and NVIDIA GPU Operator. For more information, see [Installing the Node Feature Discovery Operator](#) and [Enabling NVIDIA GPUs](#).

#### Procedure

1. In the left menu, click **Projects**.  
The **Projects** page opens.
2. Click the name of the project that you want to deploy a model in.  
A project details page opens.
3. Click the **Deployments** tab.
4. In the **Deployments** section, perform one of the following actions:
  - On the **NVIDIA NIM model serving platform** tile, click **Select NVIDIA NIM** on the tile, and then click **Deploy model**.
  - If you have previously selected the NVIDIA NIM model serving type, the **Deployments** page displays **NVIDIA model serving enabled** on the upper-right corner, along with the **Deploy model** button. To proceed, click **Deploy model**.

The **Deploy model** dialog opens.

5. Configure properties for deploying your model as follows:
  - a. In the **Model deployment name** field, enter a unique name for the deployment.
  - b. From the **NVIDIA NIM** list, select the NVIDIA NIM model that you want to deploy. For more information, see [Supported Models](#)

- c. In the **NVIDIA NIM storage size** field, specify the size of the cluster storage instance that will be created to store the NVIDIA NIM model.



#### NOTE

When resizing a PersistentVolumeClaim (PVC) backed by Amazon EBS in OpenShift AI, you may encounter **VolumeModificationRateExceeded: You've reached the maximum modification rate per volume limit**. To avoid this error, wait at least six hours between modifications per EBS volume. If you resize a PVC before the cooldown expires, the Amazon EBS CSI driver ([ebs.csi.aws.com](https://ebs.csi.aws.com)) fails with this error. This error is an Amazon EBS service limit that applies to all workloads using EBS-backed PVCs.

- d. In the **Number of model server replicas to deploy** field, specify a value.
  - e. From the **Model server size** list, select a value.
6. From the **Hardware profile** list, select a hardware profile.
  7. Optional: Click **Customize resource requests and limit** and update the following values:
    - a. In the **CPUs requests** field, specify the number of CPUs to use with your model server. Use the list beside this field to specify the value in cores or millicores.
    - b. In the **CPU limits** field, specify the maximum number of CPUs to use with your model server. Use the list beside this field to specify the value in cores or millicores.
    - c. In the **Memory requests** field, specify the requested memory for the model server in gibibytes (Gi).
    - d. In the **Memory limits** field, specify the maximum memory limit for the model server in gibibytes (Gi).
  8. Optional: In the **Model route** section, select the **Make deployed models available through an external route** checkbox to make your deployed models available to external clients.
  9. To require token authentication for inference requests to the deployed model, perform the following actions:
    - a. Select **Require token authentication**.
    - b. In the **Service account name** field, enter the service account name that the token will be generated for.
    - c. To add an additional service account, click **Add a service account** and enter another service account name.
  10. Click **Deploy**.

#### Verification

- Confirm that the deployed model is shown on the **Deployments** tab for the project, and on the **Deployments** page of the dashboard with a checkmark in the **Status** column.

#### Additional resources

- [NVIDIA NIM API reference](#)
- [Supported Models](#)

## 3.2. VIEWING NVIDIA NIM METRICS FOR A NIM MODEL

In OpenShift AI, you can observe the following NVIDIA NIM metrics for a NIM model deployed on the NVIDIA NIM model serving platform:

- GPU cache usage over time (ms)
- Current running, waiting, and max requests count
- Tokens count
- Time to first token
- Time per output token
- Request outcomes

You can specify a time range and a refresh interval for these metrics to help you determine, for example, the peak usage hours and model performance at a specified time.

### Prerequisites

- You have enabled the NVIDIA NIM model serving platform.
- You have deployed a NIM model on the NVIDIA NIM model serving platform.
- A cluster administrator has enabled metrics collection and graph generation for your deployment.
- The **disableKServeMetrics** OpenShift AI dashboard configuration option is set to its default value of **false**:

```
disableKServeMetrics: false
```

For more information about setting dashboard configuration options, see [Customizing the dashboard](#).

### Procedure

1. From the OpenShift AI dashboard navigation menu, click **Projects**.  
The **Projects** page opens.
2. Click the name of the project that contains the NIM model that you want to monitor.
3. In the project details page, click the **Deployments** tab.
4. Click the NIM model that you want to observe.
5. On the **NIM Metrics** tab, set the following options:
  - **Time range** - Specifies how long to track the metrics. You can select one of these values: 1 hour, 24 hours, 7 days, and 30 days.

- **Refresh interval** - Specifies how frequently the graphs on the metrics page are refreshed (to show the latest data). You can select one of these values: 15 seconds, 30 seconds, 1 minute, 5 minutes, 15 minutes, 30 minutes, 1 hour, 2 hours, and 1 day.

6. Scroll down to view data graphs for NIM metrics.

## Verification

The **NIM Metrics** tab shows graphs of NIM metrics for the deployed NIM model.

## Additional resources

- [NVIDIA NIM observability](#)

## 3.3. VIEWING PERFORMANCE METRICS FOR A NIM MODEL

You can observe the following performance metrics for a NIM model deployed on the NVIDIA NIM model serving platform:

- **Number of requests** - The number of requests that have failed or succeeded for a specific model.
- **Average response time (ms)** - The average time it takes a specific model to respond to requests.
- **CPU utilization (%)** - The percentage of the CPU limit per model replica that is currently utilized by a specific model.
- **Memory utilization (%)** - The percentage of the memory limit per model replica that is utilized by a specific model.

You can specify a time range and a refresh interval for these metrics to help you determine, for example, the peak usage hours and model performance at a specified time.

## Prerequisites

- You have enabled the NVIDIA NIM model serving platform.
- You have deployed a NIM model on the NVIDIA NIM model serving platform.
- A cluster administrator has enabled metrics collection and graph generation for your deployment.
- The **disableKServeMetrics** OpenShift AI dashboard configuration option is set to its default value of **false**:

```
disableKServeMetrics: false
```

For more information about setting dashboard configuration options, see [Customizing the dashboard](#).

## Procedure

1. From the OpenShift AI dashboard navigation menu, click **Projects**.  
The **Projects** page opens.

2. Click the name of the project that contains the NIM model that you want to monitor.
3. In the project details page, click the **Deployments** tab.
4. Click the NIM model that you want to observe.
5. On the **Endpoint performance** tab, set the following options:
  - **Time range** - Specifies how long to track the metrics. You can select one of these values: 1 hour, 24 hours, 7 days, and 30 days.
  - **Refresh interval** - Specifies how frequently the graphs on the metrics page are refreshed to show the latest data. You can select one of these values: 15 seconds, 30 seconds, 1 minute, 5 minutes, 15 minutes, 30 minutes, 1 hour, 2 hours, and 1 day.
6. Scroll down to view data graphs for performance metrics.

## Verification

The **Endpoint performance** tab shows graphs of performance metrics for the deployed NIM model.

## CHAPTER 4. MAKING INFERENCE REQUESTS TO DEPLOYED MODELS

When you deploy a model, it is available as a service that you can access with API requests. This allows you to get predictions from your model based on the data you provide in the request.

### 4.1. ACCESSING THE AUTHENTICATION TOKEN FOR A DEPLOYED MODEL

If you secured your model inference endpoint by enabling token authentication, you must know how to access your authentication token so that you can specify it in your inference requests.

#### Prerequisites

- You have logged in to Red Hat OpenShift AI.
- You have deployed a model by using the model serving platform.

#### Procedure

1. From the OpenShift AI dashboard, click **Projects**.  
The **Projects** page opens.
2. Click the name of the project that contains your deployed model.  
A project details page opens.
3. Click the **Deployments** tab.
4. In the **Deployments** list, expand the section for your model.  
Your authentication token is shown in the **Token authentication** section, in the **Token secret** field.
5. Optional: To copy the authentication token for use in an inference request, click the **Copy** button (  ) next to the token value.

### 4.2. ACCESSING THE INFERENCE ENDPOINT FOR A DEPLOYED MODEL

To make inference requests to your deployed model, you must know how to access the inference endpoint that is available.

For a list of paths to use with the supported runtimes and example commands, see [Inference endpoints](#).

#### Prerequisites

- You have logged in to Red Hat OpenShift AI.
- You have deployed a model by using the model serving platform.
- If you enabled token authentication for your deployed model, you have the associated token value.

## Procedure

1. From the OpenShift AI dashboard, click **AI hub** → **Deployments**.  
The inference endpoint for the model is shown in the **Inference endpoints** field.
2. Depending on what action you want to perform with the model (and if the model supports that action), copy the inference endpoint and then add a path to the end of the URL.
3. Use the endpoint to make API requests to your deployed model.

## Additional resources

- [Caikit API documentation](#)
- [OpenVINO KServe-compatible REST API documentation](#)
- [OpenAI API documentation](#)

## 4.3. MAKING INFERENCE REQUESTS TO MODELS DEPLOYED ON THE MODEL SERVING PLATFORM

When you deploy a model by using the model serving platform, the model is available as a service that you can access using API requests. This enables you to return predictions based on data inputs. To use API requests to interact with your deployed model, you must know the inference endpoint for the model.

In addition, if you secured your inference endpoint by enabling token authentication, you must know how to access your authentication token so that you can specify this in your inference requests.

## 4.4. INFERENCE ENDPOINTS

These examples show how to use inference endpoints to query the model.



### NOTE

If you enabled token authentication when deploying the model, add the **Authorization** header and specify a token value.

### 4.4.1. Caikit TGIS ServingRuntime for KServe

- **:443/api/v1/task/text-generation**
- **:443/api/v1/task/server-streaming-text-generation**

#### Example command

```
curl --json '{"model_id": "<model_name__>", "inputs": "<text>"}'
https://<inference_endpoint_url>:443/api/v1/task/server-streaming-text-generation -H 'Authorization:
Bearer <token>'
```

### 4.4.2. OpenVINO Model Server

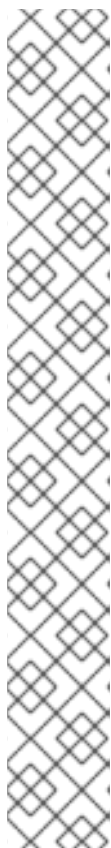
- **/v2/models/<model-name>/infer**

## Example command

```
curl -ks <inference_endpoint_url>/v2/models/<model_name>/infer -d '{ "model_name": "  
<model_name>", "inputs": [{ "name": "<name_of_model_input>", "shape": [<shape>], "datatype": "  
<data_type>", "data": [<data>] } ]}' -H 'Authorization: Bearer <token>'
```

### 4.4.3. vLLM NVIDIA GPU ServingRuntime for KServe

- **:443/version**
- **:443/docs**
- **:443/v1/models**
- **:443/v1/chat/completions**
- **:443/v1/completions**
- **:443/v1/embeddings**
- **:443/tokenize**
- **:443/detokenize**



#### NOTE

- The vLLM runtime is compatible with the OpenAI REST API.
- To use the embeddings inference endpoint in vLLM, you must use an embeddings model that the vLLM supports. You cannot use the embeddings endpoint with generative models. For more information, see [Supported embeddings models in vLLM](#).
- As of vLLM v0.5.5, you must provide a chat template while querying a model using the **/v1/chat/completions** endpoint. If your model does not include a predefined chat template, you can use the **chat-template** command-line parameter to specify a chat template in your custom vLLM runtime, as shown in the example. Replace **<CHAT\_TEMPLATE>** with the path to your template.

```
containers:
  - args:
    - --chat-template=<CHAT_TEMPLATE>
```

You can use the chat templates that are available as **.jinja** files [here](#) or with the vLLM image under **/opt/app-root/template**. For more information, see [Chat templates](#).

As indicated by the paths shown, the model serving platform uses the HTTPS port of your OpenShift router (usually port 443) to serve external API requests.

## Example command

```
curl -v https://<inference_endpoint_url>:443/v1/chat/completions -H "Content-Type: application/json" -
```

```
d '{ "messages": [{ "role": "<role>", "content": "<content>" }] } -H 'Authorization: Bearer <token>'
```

#### 4.4.4. vLLM Intel Gaudi Accelerator ServingRuntime for KServe

See [vLLM NVIDIA GPU ServingRuntime for KServe](#).

#### 4.4.5. vLLM AMD GPU ServingRuntime for KServe

See [vLLM NVIDIA GPU ServingRuntime for KServe](#).

#### 4.4.6. vLLM Spyre AI Accelerator ServingRuntime for KServe



##### IMPORTANT

Support for IBM Spyre AI Accelerators on x86 is currently available in Red Hat OpenShift AI 3.4 as a Technology Preview feature. Technology Preview features are not supported with Red Hat production service level agreements (SLAs) and might not be functionally complete. Red Hat does not recommend using them in production. These features provide early access to upcoming product features, enabling customers to test functionality and provide feedback during the development process.

For more information about the support scope of Red Hat Technology Preview features, see [Technology Preview Features Support Scope](#).

You can serve models with IBM Spyre AI accelerators on x86 by using the **vLLM Spyre AI Accelerator ServingRuntime for KServe** runtime. To use the runtime, you must install the Spyre Operator and configure a hardware profile. For more information, see [Spyre operator image](#) and [Working with hardware profiles](#).

#### 4.4.7. vLLM Spyre s390x ServingRuntime for KServe

You can serve models with IBM Spyre AI accelerators on IBM Z (s390x architecture) by using the **vLLM Spyre s390x ServingRuntime for KServe** runtime. To use the runtime, you must install the Spyre Operator and configure a hardware profile. For more information, see [Spyre operator image](#) and [Working with hardware profiles](#).

#### 4.4.8. vLLM Spyre ppc64le ServingRuntime for KServe

You can serve models with IBM Spyre AI accelerators on IBM Power (ppc64le architecture) by using the **vLLM Spyre ppc64le ServingRuntime for KServe** runtime. To use the runtime, you must install the Spyre Operator and configure a hardware profile. For more information, see [Spyre operator image](#) and [Working with hardware profiles](#).

#### 4.4.9. NVIDIA Triton Inference Server

REST endpoints

- **v2/models/[versions/<model\_version>]/infer**
- **v2/models/<model\_name>[/versions/<model\_version>]**
- **v2/health/ready**

- **v2/health/live**
- **v2/models/<model\_name>[/versions/]/ready**
- **v2**

#### Example command

```
curl -ks <inference_endpoint_url>/v2/models/<model_name>/infer -d '{ "model_name": "  
<model_name>", "inputs": [{ "name": "<name_of_model_input>", "shape": [<shape>], "datatype": "  
<data_type>", "data": [<data>] } ]}' -H 'Authorization: Bearer <token>'
```

#### gRPC endpoints

- **:443 inference.GRPCInferenceService/ModelInfer**
- **:443 inference.GRPCInferenceService/ModelReady**
- **:443 inference.GRPCInferenceService/ModelMetadata**
- **:443 inference.GRPCInferenceService/ServerReady**
- **:443 inference.GRPCInferenceService/ServerLive**
- **:443 inference.GRPCInferenceService/ServerMetadata**

#### Example command

```
grpcurl -cacert ./openshift_ca_istio_knative.crt -proto ./grpc_predict_v2.proto -d @ -H "Authorization:  
Bearer <token>" <inference_endpoint_url>:443 inference.GRPCInferenceService/ModelMetadata
```

### 4.4.10. MLServer ServingRuntime for KServe



#### IMPORTANT

MLServer ServingRuntime for KServe is currently available in Red Hat OpenShift AI as a Technology Preview feature. Technology Preview features are not supported with Red Hat production service level agreements (SLAs) and might not be functionally complete. Red Hat does not recommend using them in production. These features provide early access to upcoming product features, enabling customers to test functionality and provide feedback during the development process.

For more information about the support scope of Red Hat Technology Preview features, see [Technology Preview Features Support Scope](#).



#### NOTE

HTTP requests to the MLServer ServingRuntime for KServe require the **Content-Type: application/json** header.

#### REST endpoints

- **/v2**

- **/v2/health/live**
- **/v2/health/ready**
- **/v2/models/{model\_name}**
- **/v2/models/{model\_name}/infer**
- **/v2/models/{model\_name}/ready**

### Example command

```
curl -ks <inference_endpoint_url>/v2/models/<model_name>/infer -H "Content-Type: application/json" -d '{ "inputs": [{ "name": "<name_of_model_input>", "shape": [<shape>], "datatype": "<data_type>", "data": [<data>] } ]}' -H 'Authorization: Bearer <token>'
```

For detailed configuration guidance, see [Deploying models by using the MLServer runtime](#) .

### 4.4.11. Additional resources

- [Caikit API documentation](#)
- [OpenVINO KServe-compatible REST API documentation](#)
- [OpenAI API documentation](#)
- [Open Inference Protocol](#)
- [Supported model-serving runtimes](#) .